服务器逻辑层设计和消息完善

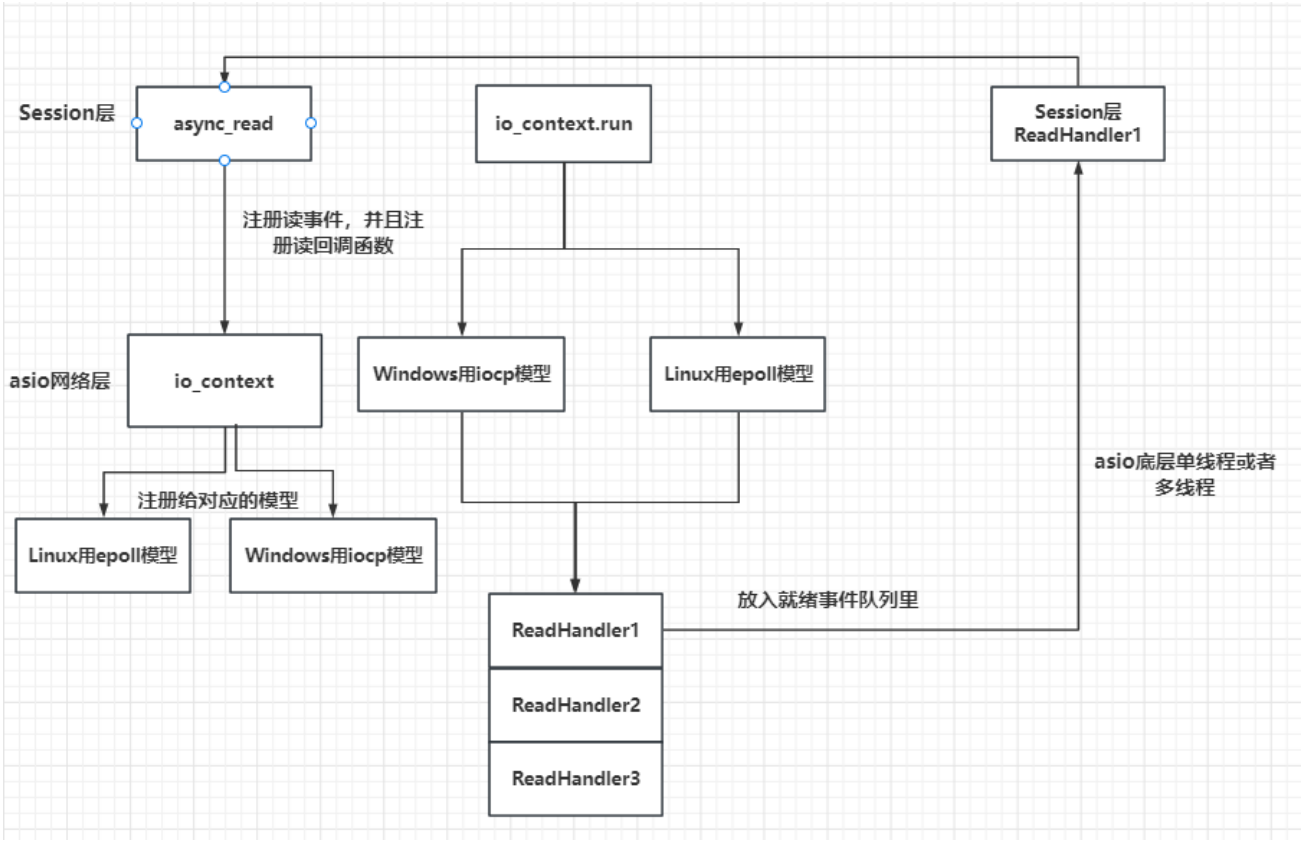
Posted on 2023-06-01 20:13:50 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2QbUASZ5jV8jgKYFngRHrG8pu7z)

简介

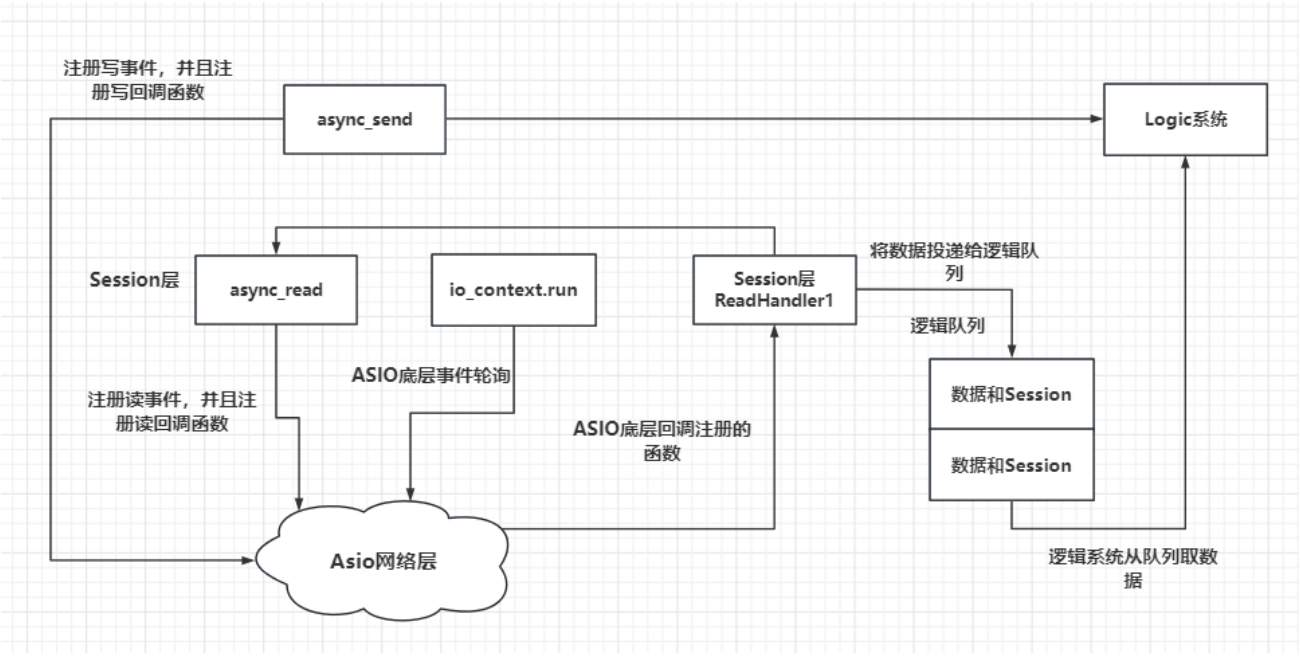
本文概述基于 `boost::asio` 实现的服务器逻辑层结构，并且完善之前设计的消息结构。因为为了简化粘包处理，我们简化了发送数据的结构,这次我们给出完整的消息设计，以及服务器架构设计。

服务器架构设计

之前我们设计了Session(会话层)，并且给大家讲述了Asio底层的通信过程，如下图

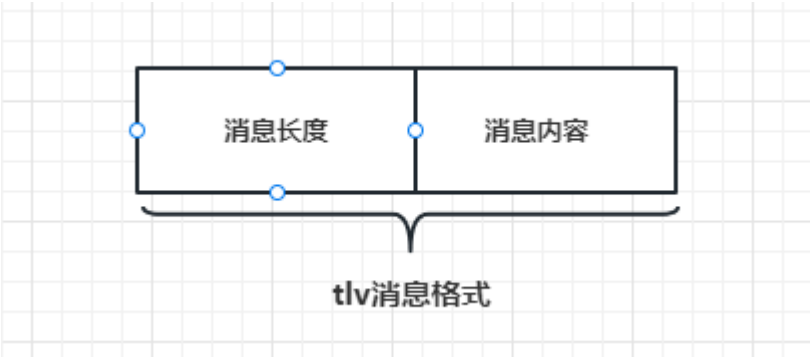


我们接下来要设计的服务器结构是这样的

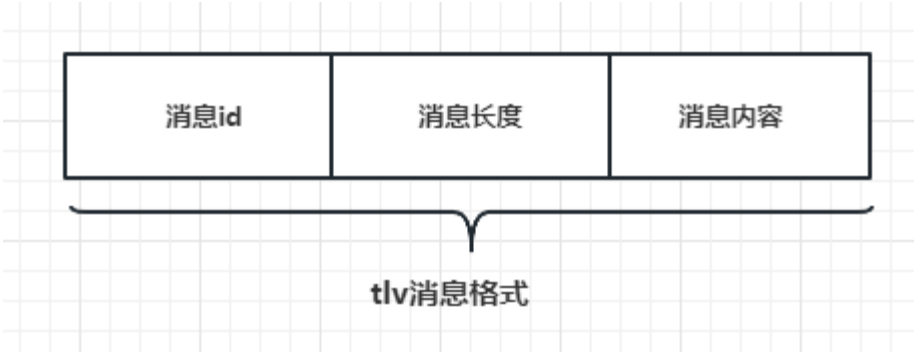


消息头完善

我们之前的消息头仅包含数据域的长度，但是要进行逻辑处理，就需要传递一个id字段表示要处理的消息id，当然可以不在包头传id字段，将id序列化到消息体也是可以的，但是我们为了便于处理也便于回调逻辑层对应的函数，最好是将id写入包头。
之前我们设计的消息结构是这样的



现在将其完善为如下的样子



为了减少耦合和歧义，我们重新设计消息节点。

`MsgNode` 表示消息节点的基类，头部的消息用这个结构存储。

`RecvNode` 表示接收消息的节点。

`SendNode` 表示发送消息的节点。

我们将上述结构定义在MsgNode.h中

```
1. class MsgNode
2. {
3. public:
4.     MsgNode(short max_len) :_total_len(max_len), _cur_len(0) {
5.         _data = new char[_total_len + 1]();
6.         _data[_total_len] = '\0';
7.     }
8.
9.     ~MsgNode() {
10.        std::cout << "destruct MsgNode" << endl;
11.        delete[] _data;
12.    }
13.
14.    void Clear() {
15.        ::memset(_data, 0, _total_len);
16.        _cur_len = 0;
17.    }
18.
19.    short _cur_len;
20.    short _total_len;
21.    char* _data;
22. };
23.
24. class RecvNode :public MsgNode {
25. public:
26.     RecvNode(short max_len, short msg_id);
27. private:
28.     short _msg_id;
29. };
30.
31. class SendNode:public MsgNode {
32. public:
33.     SendNode(const char* msg,short max_len, short msg_id);
34. private:
35.     short _msg_id;
36. };
```

实现如下

```
#include "MsgNode.h"
RecvNode::RecvNode(short max_len, short msg_id):MsgNode(max_len),
_msg_id(msg_id){

}


SendNode::SendNode(const char* msg, short max_len, short msg_id):MsgNode(max_len + HEAD_TOTAL_LEN)
, _msg_id(msg_id){
    //先发送id, 转为网络字节序
    short msg_id_host = boost::asio::detail::socket_ops::host_to_network_short(msg_id);
    memcpy(_data, &msg_id_host, HEAD_ID_LEN);
    //转为网络字节序
    short max_len_host = boost::asio::detail::socket_ops::host_to_network_short(max_len);
    memcpy(_data + HEAD_ID_LEN, &max_len_host, HEAD_DATA_LEN);
    memcpy(_data + HEAD_ID_LEN + HEAD_DATA_LEN, msg, max_len);
}
```

`SendNode` 发送节点构造时，先将id转为网络字节序，然后写入 `_data` 数据域。
然后将要发送数据的长度转为大端字节序，写入 `_data` 数据域，注意要偏移 `HEAD_ID_LEN` 长度。
最后将要发送的数据 `msg` 写入 `_data` 数据域，注意要偏移 `HEAD_ID_LEN` + `HEAD_DATA_LEN`

Session类改写

因为消息结构改变了，所以我们接收和发送数据的逻辑要做对应的修改，我们先修改Session类中收发消息结构如下

```
    std::queue<shared_ptr<MsgNode> > _send_que;
    std::mutex _send_lock;
    //收到的消息结构
    std::shared_ptr<MsgNode> _recv_msg_node;
    bool _b_head_parse;
    //收到的头部结构
    std::shared_ptr<MsgNode> _recv_head_node;
```

因为头部数据只为4字节，所以我们在Session的构造函数中创建头部节点时选

择 `HEAD_TOTAL_LEN` (4字节)大小。

```
1. CSession::CSession(boost::asio::io_context& io_context, CServer* server):
2.     _socket(io_context), _server(server), _b_close(false),_b_head_parse(false){
3.     boost::uuids::uuid  a_uuid = boost::uuids::random_generator()();
4.     _uuid = boost::uuids::to_string(a_uuid);
5.     _recv_head_node = make_shared<MsgNode>(HEAD_TOTAL_LEN);
6. }
```

发送时我们构造发送节点，放到队列中即可

```
1. void CSession::Send(char* msg, short max_length, short msgid) {
2.     std::lock_guard<std::mutex> lock(_send_lock);
3.     int send_que_size = _send_que.size();
4.     if (send_que_size > MAX_SENDQUE) {
5.         std::cout << "session: " << _uuid << " send que fulled, size is " << MAX_SENDQUE << endl;
6.         return;
7.     }
8.
9.     _send_que.push(make_shared<SendNode>(msg, max_length, msgid));
10.    if (send_que_size>0) {
11.        return;
12.    }
13.    auto& msgnode = _send_que.front();
14.    boost::asio::async_write(_socket, boost::asio::buffer(msgnode->_data, msgnode->_total_le
    n),
15.        std::bind(&CSession::HandleWrite, this, std::placeholders::_1, SharedSelf()));
16. }
```

当然我们也实现了一个重载版本

```
1. void CSession::Send(std::string msg, short msgid) {
2.     std::lock_guard<std::mutex> lock(_send_lock);
3.     int send_que_size = _send_que.size();
4.     if (send_que_size > MAX_SENDQUE) {
5.         std::cout << "session: " << _uuid << " send que fulled, size is " << MAX_SENDQUE << endl;
6.         return;
7.     }
8.
9.     _send_que.push(make_shared<SendNode>(msg.c_str(), msg.length(), msgid));
10.    if (send_que_size > 0) {
11.        return;
12.    }
13.    auto& msgnode = _send_que.front();
14.    boost::asio::async_write(_socket, boost::asio::buffer(msgnode->_data, msgnode->_total_le
    n),
15.        std::bind(&CSession::HandleWrite, this, std::placeholders::_1, SharedSelf()));
16. }
```

在接收数据时我们解析头部也要解析id字段

```
1. void CSession::HandleRead(const boost::system::error_code& error, size_t  bytes_transferred, std::shared_ptr<CSession> shared_self){
2.     try {
3.         if (!error) {
4.             //已经移动的字符数
5.             int copy_len = 0;
6.             while (bytes_transferred > 0) {
7.                 if (!_b_head_parse) {
8.                     //收到的数据不足头部大小
9.                     if (bytes_transferred + _recv_head_node->_cur_len < HEAD_TOTAL_LEN) {
10.                        memcpy(_recv_head_node->_data + _recv_head_node->_cur_len, _data + copy_len, bytes_transferred);
11.                        _recv_head_node->_cur_len += bytes_transferred;
12.                        ::memset(_data, 0, MAX_LENGTH);
13.                        _socket.async_read_some(boost::asio::buffer(_data, MAX_LENGTH),
14.                            std::bind(&CSession::HandleRead, this, std::placeholders::_1, std::placeholders::_2, shared_self));
15.                        return;
16.                    }
17.                    //收到的数据比头部多
18.                    //头部剩余未复制的长度
19.                    int head_remain = HEAD_TOTAL_LEN - _recv_head_node->_cur_len;
20.                    memcpy(_recv_head_node->_data + _recv_head_node->_cur_len, _data + copy_len, head_remain);
21.                    //更新已处理的data长度和剩余未处理的长度
22.                    copy_len += head_remain;
23.                    bytes_transferred -= head_remain;
24.                    //获取头部MSGID数据
25.                    short msg_id = 0;
26.                    memcpy(&msg_id, _recv_head_node->_data, HEAD_ID_LEN);
27.                    //网络字节序转化为本地字节序
28.                    msg_id = boost::asio::detail::socket_ops::network_to_host_short(msg_id);
29.                    std::cout << "msg_id is " << msg_id << endl;
30.                    //id非法
31.                    if (msg_id > MAX_LENGTH) {
32.                        std::cout << "invalid msg_id is " << msg_id << endl;
33.                        _server->ClearSession(_uuid);
34.                        return;
35.                    }
36.                    short msg_len = 0;
37.                    memcpy(&msg_len, _recv_head_node->_data+HEAD_ID_LEN, HEAD_DATA_LEN);
38.                    //网络字节序转化为本地字节序
```

```
39.             msg_len = boost::asio::detail::socket_ops::network_to_host_short(msg_len);
40.             std::cout << "msg_len is " << msg_len << endl;
41.             //id非法
42.             if (msg_len > MAX_LENGTH) {
43.                 std::cout << "invalid data length is " << msg_len << endl;
44.                 _server->ClearSession(_uuid);
45.                 return;
46.             }
47.
48.             _recv_msg_node = make_shared<RecvNode>(msg_len, msg_id);
49.
50.             //消息的长度小于头部规定的长度，说明数据未收全，则先将部分消息放到接收节点里
51.             if (bytes_transferred < msg_len) {
52.                 memcpy(_recv_msg_node->_data + _recv_msg_node->_cur_len, _data + copy_len, bytes_transferred);
53.                 _recv_msg_node->_cur_len += bytes_transferred;
54.                 ::memset(_data, 0, MAX_LENGTH);
55.                 _socket.async_read_some(boost::asio::buffer(_data, MAX_LENGTH),
56.                     std::bind(&CSession::HandleRead, this, std::placeholders::_1, std::placeholders::_2, shared_self));
57.                 //头部处理完成
58.                 _b_head_parse = true;
59.                 return;
60.             }
61.
62.             memcpy(_recv_msg_node->_data + _recv_msg_node->_cur_len, _data + copy_len, msg_len);
63.             _recv_msg_node->_cur_len += msg_len;
64.             copy_len += msg_len;
65.             bytes_transferred -= msg_len;
66.             _recv_msg_node->_data[_recv_msg_node->_total_len] = '\0';
67.             //cout << "receive data is " << _recv_msg_node->_data << endl;
68.             //此处可以调用Send发送测试
69.             Json::Reader reader;
70.             Json::Value root;
71.             reader.parse(std::string(_recv_msg_node->_data, _recv_msg_node->_total_len), root);
72.             std::cout << "recevie msg id  is " << root["id"].asInt() << " msg data is "
73.                 << root["data"].asString() << endl;
74.             root["data"] = "server has received msg, msg data is " + root["data"].asString();
75.             std::string return_str = root.toStyledString();
```

```
76.                 Send(return_str, root["id"].asInt());
77.                 //继续轮询剩余未处理数据
78.                 _b_head_parse = false;
79.                 _recv_head_node->Clear();
80.                 if (bytes_transferred <= 0) {
81.                     ::memset(_data, 0, MAX_LENGTH);
82.                     _socket.async_read_some(boost::asio::buffer(_data, MAX_LENGTH),
83.                         std::bind(&CSession::HandleRead, this, std::placeholders::_1, std::placeh
    olders::_2, shared_self));
84.                     return;
85.                 }
86.                 continue;
87.             }
88.
89.             //已经处理完头部，处理上次未接受完的消息数据
90.             //接收的数据仍不足剩余未处理的
91.             int remain_msg = _recv_msg_node->_total_len - _recv_msg_node->_cur_len;
92.             if (bytes_transferred < remain_msg) {
93.                 memcpy(_recv_msg_node->_data + _recv_msg_node->_cur_len, _data + copy_le
    n, bytes_transferred);
94.                 _recv_msg_node->_cur_len += bytes_transferred;
95.                 ::memset(_data, 0, MAX_LENGTH);
96.                 _socket.async_read_some(boost::asio::buffer(_data, MAX_LENGTH),
97.                     std::bind(&CSession::HandleRead, this, std::placeholders::_1, std::placehold
    ers::_2, shared_self));
98.                 return;
99.             }
100.            memcpy(_recv_msg_node->_data + _recv_msg_node->_cur_len, _data + copy_len, r
    emain_msg);
101.            _recv_msg_node->_cur_len += remain_msg;
102.            bytes_transferred -= remain_msg;
103.            copy_len += remain_msg;
104.            _recv_msg_node->_data[_recv_msg_node->_total_len] = '\0';
105.            //cout << "receive data is " << _recv_msg_node->_data << endl;
106.                //此处可以调用Send发送测试
107.            Json::Reader reader;
108.            Json::Value root;
109.            reader.parse(std::string(_recv_msg_node->_data, _recv_msg_node->_total_len), roo
    t);
110.            std::cout << "recevie msg id  is " << root["id"].asInt() << " msg data is "
111.                << root["data"].asString() << endl;
112.            root["data"] = "server has received msg, msg data is " + root["data"].asString();
```

```
113.             std::string return_str = root.toStyledString();
114.             Send(return_str, root["id"].asInt());
115.             //继续轮询剩余未处理数据
116.             _b_head_parse = false;
117.             _recv_head_node->Clear();
118.             if (bytes_transferred <= 0) {
119.                 ::memset(_data, 0, MAX_LENGTH);
120.                 _socket.async_read_some(boost::asio::buffer(_data, MAX_LENGTH),
121.                     std::bind(&CSession::HandleRead, this, std::placeholders::_1, std::placehold
    ers::_2, shared_self));
122.                 return;
123.             }
124.             continue;
125.         }
126.     }
127.     else {
128.         std::cout << "handle read failed, error is " << error.what() << endl;
129.         Close();
130.         _server->ClearSession(_uuid);
131.     }
132. }
133. catch (std::exception& e) {
134.     std::cout << "Exception code is " << e.what() << endl;
135. }
136. }
```

先解析头部id，再解析长度，然后根据id和长度构造消息节点，copy剩下的消息体, 把上面代码中处理消息头的逻辑截取如下

```
1.  //获取头部MSGID数据
2.  short msg_id = 0;
3.  memcpy(&msg_id, _recv_head_node->_data, HEAD_ID_LEN);
4.  //网络字节序转化为本地字节序
5.  msg_id = boost::asio::detail::socket_ops::network_to_host_short(msg_id);
6.  std::cout << "msg_id is " << msg_id << endl;
7.  //id非法
8.  if (msg_id > MAX_LENGTH) {
9.      std::cout << "invalid msg_id is " << msg_id << endl;
10.     _server->ClearSession(_uuid);
11.     return;
12. }
13.
14. short msg_len = 0;
15. memcpy(&msg_len, _recv_head_node->_data+HEAD_ID_LEN, HEAD_DATA_LEN);
16. //网络字节序转化为本地字节序
17. msg_len = boost::asio::detail::socket_ops::network_to_host_short(msg_len);
18. std::cout << "msg_len is " << msg_len << endl;
19. //id非法
20. if (msg_len > MAX_LENGTH) {
21.     std::cout << "invalid data length is " << msg_len << endl;
22.     _server->ClearSession(_uuid);
23.     return;
24. }
25.
26. _recv_msg_node = make_shared<RecvNode>(msg_len, msg_id);
```

其余的没什么变动。

总结

本文介绍了服务器逻辑和网络层的设计，并且基于这个架构，完善了消息发送结构，下一篇带着大家设计逻辑类和逻辑队列。

源码链接https://gitee.com/secondtonone1/boostasio-learn (https://gitee.com/secondtonone1/boostasio-learn)

浏览(3178) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(577) (JavaScript:void(0);)

热门评论

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



恋恋风辰

北京 朝阳



扫一扫上面的二维码图案，加我为朋友。

个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。

京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)

京ICP备2021033081号 (https://beian.miit.gov.cn/)